Making Money by Being Wrong

Stock Trading From Social Media Sentiment Analysis

Art Steinmetz

Goals

- Learn how to manage large datasets with R and duckplyr.
- Learn how to potentially profit from social media posters being reliably wrong.

Who I Am

- Retired Investment Professional (Global Macro Fixed Income)
- Fund Board Member at BlackRock
- Consultant for Posit
- Data Science Hobbyist
- Blogger at `outsiderdata.net`

Background

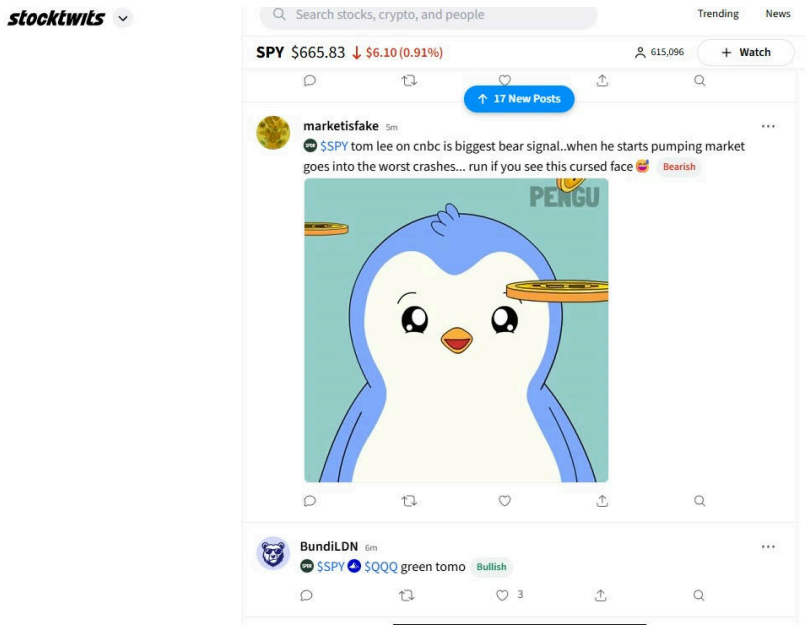
- Picking stocks that will go up is hard.
- Picking stocks that will go down also is hard.
- Being skilled at either could make you a lot of money.
- **But what if we could find people who are reliably wrong?**

Social Media Sentiment Analysis

- A recent academic paper presented a huge dataset. *"StockTwits: Comprehensive records of a financial social media platform from 2008 to 2022."[1]*
- The complete dataset is freely available on AWS S3, over 500 million posts!
- Of the posters, the researchers found "**a significant percentage consistently perform statistically better or worse than guessing, especially at longer timescales.**"

What is StockTwits?

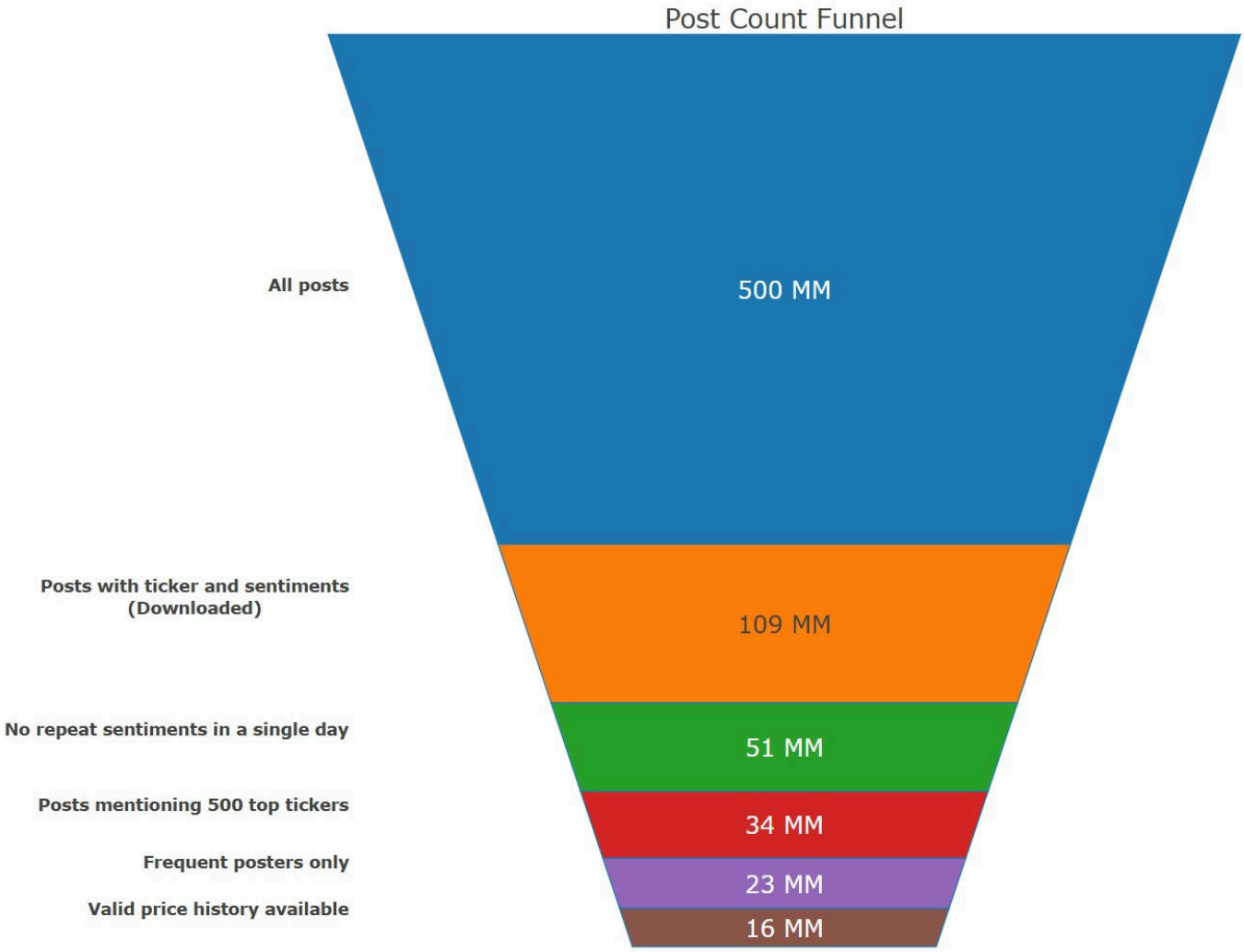
- This is what Copilot suggested I say: “StockTwits is a social media platform designed for sharing ideas between investors, traders, and entrepreneurs.””
- This is what I say: “StockTwits is Twitter for randos to hype meme stocks…with memes.”



The Plan

- Download symbol_sentiment CSV files of the StockTwits dataset from AWS S3.
- Filter the dataset down to useful posts.
- Download matching stock price data from Yahoo Finance.
- Calculate 7-day forward returns after each message.
- Filter for statistically significant batting averages.
- Develop a trading strategy.

From 500 Million Down To 16 Million Posts



Managing The Data

- The R dplyr package is great a manipulating large data frames but is designed for in-memory usage.
- Manipulating very large data sets is best done with a database package that works with connections to those data sets, wherever they reside.
- We will use the super-fast **DuckDB** for R (Python users should consider Polars).

Managing The Data (cont.)

- DuckDB has a dplyr frontend called **duckplyr** where most dplyr verbs are implemented.
- We'll save and retrieve (connect to) files in **Parquet** format which, like duckDB, is column-oriented.
- **Differences in syntax are almost invisible but speed advantage is enormous.**

duckDB Is SQL-based....

```
# Find the largest sepals/petals in the Iris data set
library(duckdb)

con <- dbConnect(duckdb())
duckdb_register(con, "iris", iris)

query <- r'(
    SELECT count(*) AS num_observations,
    max("Sepal.Width") AS max_width,
    max("Petal.Length") AS max_petal_length
    FROM iris
    WHERE "Sepal.Length" > 5
    GROUP BY ALL
    )'

dbGetQuery(con, query)
```

... But You Don't Have To Learn SQL

```
# Find the largest sepals/petals in the Iris data set using duckplyr
library("duckplyr")

iris |>
    filter(Sepal.Length > 5) |>
    summarize(.by = Species,
        num_observations = n(),
        max_width = max(Sepal.Width),
        max_petal_length = max(Petal.Length),
        na.rm = TRUE) |>
    collect()
```

duckplyr Key Points

- Chains of multiple verbs are only evaluated when `collect()` or `as_tibble()` is called, even when the data remains on disk and not in memory.
- Not all verbs are implemented, but duckplyr automatically falls back to dplyr when needed.
- Have a strategy around when to "materialize" data.

Downloading the Data is Easy!

- We are only interested in files containing the user ID, timestamp, symbol and sentiment columns.
- This 100-million-row subset is stored as 34 CSV files in an AWS S3 bucket.
- Fetching took about 25 seconds per file on my machine.
- Once downloaded, we can treat the entire dataset on disk as a single data frame.

Downloading the Data is Easy (code)

```
library(tidyverse)
library(duckplyr)

db_exec("INSTALL httpfs;")
db_exec("LOAD httpfs;")
db_exec("SET s3_region='us-west-2';")

base_url <- "https://stocktwits-nyu.s3.amazonaws.com/dataset/v1/data/csv/"
dataset <- "symbol_sentiments"
file_nums <- sprintf("%02d", 0:33)
S3_file_list <- paste0(base_url, dataset, "/",dataset, "_", file_nums, ".csv")
local_file_list <- paste0("data/", dataset, "_", file_nums, ".parquet")

for (n in 1:length(S3_file_list)) {
  df <- read_csv_duckdb(S3_file_list[n]) |>
    compute_parquet(local_file_list[n]) }

# ALL SET! Read all parquet files and off you go - instantly!
sentiments <- read_parquet_duckdb(local_file_list)
```

Understand what materialization means

```
# This gets the answer almost instantly.
sentiments |> summarise(total_rows = n())
# A duckplyr data frame: 1 variable
#   total_rows
#     <int>
#1 106105966

# This is bad
sentiments |> nrow()
# Error:
# ! Materialization would result in more than 250000 rows.
# Use `collect()` or `as_tibble()` to materialize.
```

No More Code Today. Just *RESULTS!*

- Refer to my GitHub repository for all the analytic code and details.
- https://github.com/apsteinmetz/stocktwits

The Cleaned Data

Sample StockTwits Posts After Cleaning and Filtering

user_id	ticker	date	bullish
3593665	XPEV	2020-11-19	TRUE
1678424	F	2022-01-21	TRUE
3254854	SPY	2021-12-20	TRUE
32114	USO	2017-05-09	TRUE
1540938	ENDPQ	2022-08-11	TRUE
1210343	ACRX	2018-07-30	TRUE
3625032	NNDM	2021-02-03	TRUE
678548	TSLA	2017-03-03	FALSE
1885910	CENN	2021-04-26	TRUE
2846728	TWTR	2021-03-30	FALSE

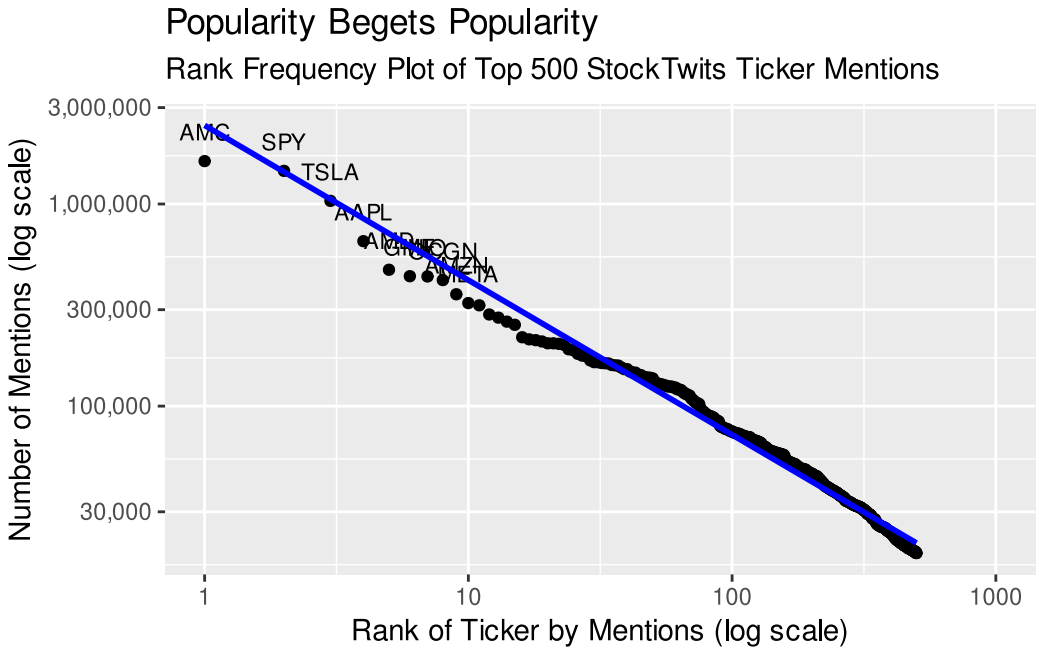
What are the most popular Tickers?

No surprise, megacaps, meme stocks, and SPY.

Most Popular StockTwits Tickers

ticker	count	start_date	end_date
AMC	1,630,346	2013-12-17	2022-12-31
SPY	1,458,864	2010-06-02	2022-12-31
TSLA	1,036,455	2010-09-16	2022-12-31
AAPL	654,935	2010-06-03	2022-12-31
AMD	472,878	2010-09-04	2022-12-31
GME	439,350	2010-08-21	2022-12-31
NIO	438,048	2012-12-20	2022-12-31
OCGN	420,353	2014-12-15	2022-12-31
AMZN	357,618	2010-06-02	2022-12-31
META	323,491	2011-01-09	2022-12-31

Zipf's Law - Stocktwits Like to Pile On



Get Prices For Most Popular Tickers

- We the `yahoofinancer` package to download historical OHLC prices from Yahoo Finance.
- Other download packages are available, e.g. `quantmod`, `tidyquant`, `yfR`.
- Some tickers don't have valid prices histories aligned with our sentiment data. We'll ignore them.

Build Track Records

- Map each post to the adjusted close price on the post date and the price 7 days later.
- Test whether the return and return vs. SPY align with sentiment

Sample Track Records

user_id	ticker	date	bullish	win_absolute	win_vs_SPY
3593665	XPEV	2020-11-19	TRUE	TRUE	TRUE
1678424	F	2022-01-21	TRUE	FALSE	FALSE
3254854	SPY	2021-12-20	TRUE	TRUE	TRUE
32114	USO	2017-05-09	TRUE	TRUE	TRUE
3625032	NNDM	2021-02-03	TRUE	TRUE	FALSE

Calculate Batting Averages

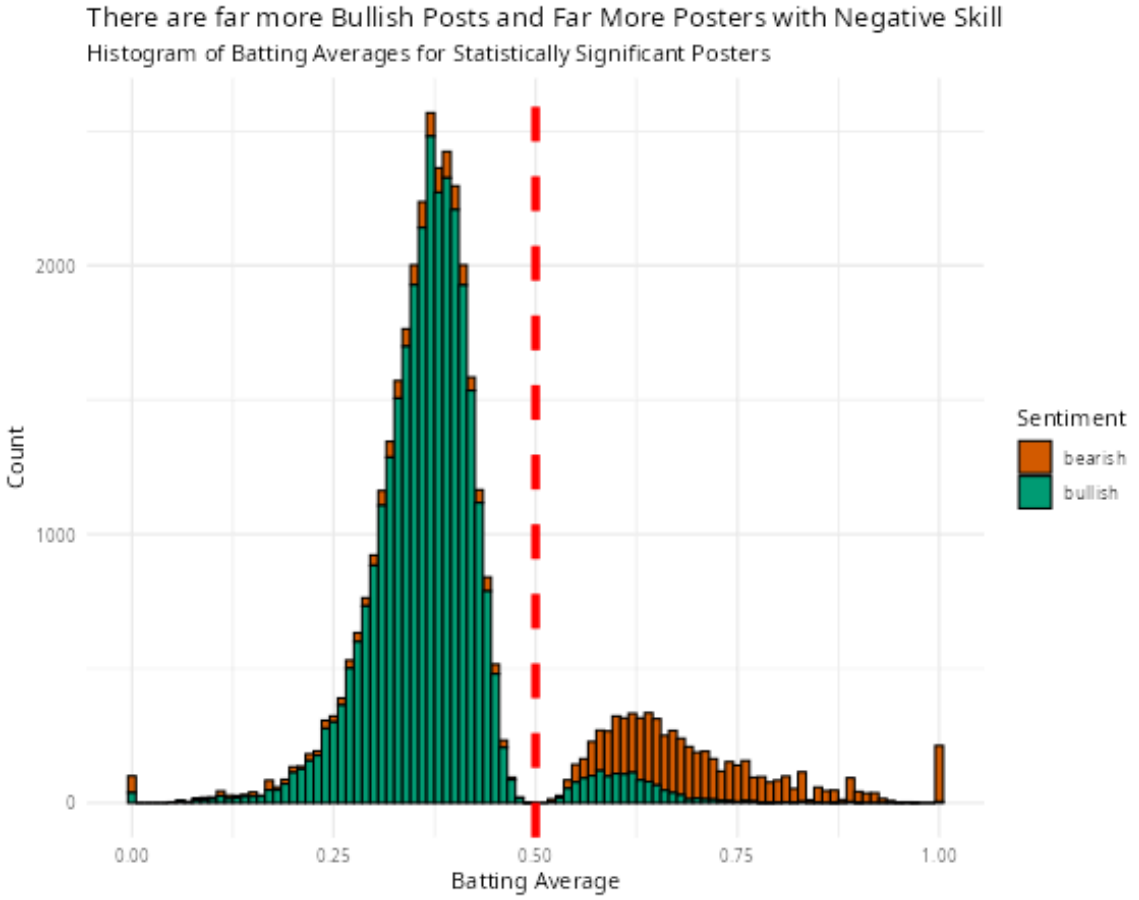
- For each ticker, calculate the batting average (percent correct) for buy and sell posts.

Sample Batting Averages

user_id	total	wins	win_rate
3055342	51	24	47%
3354926	181	81	45%
260786	100	32	32%
2964764	123	53	43%
3157678	73	29	40%

Stocktwits Are Usually Wrong

- Of 68,183 posters with track records, 37,767 had statistically significant batting averages at the 5% level ($p < 0.05$).
- Of those, the alpha direction was negative for 83%.

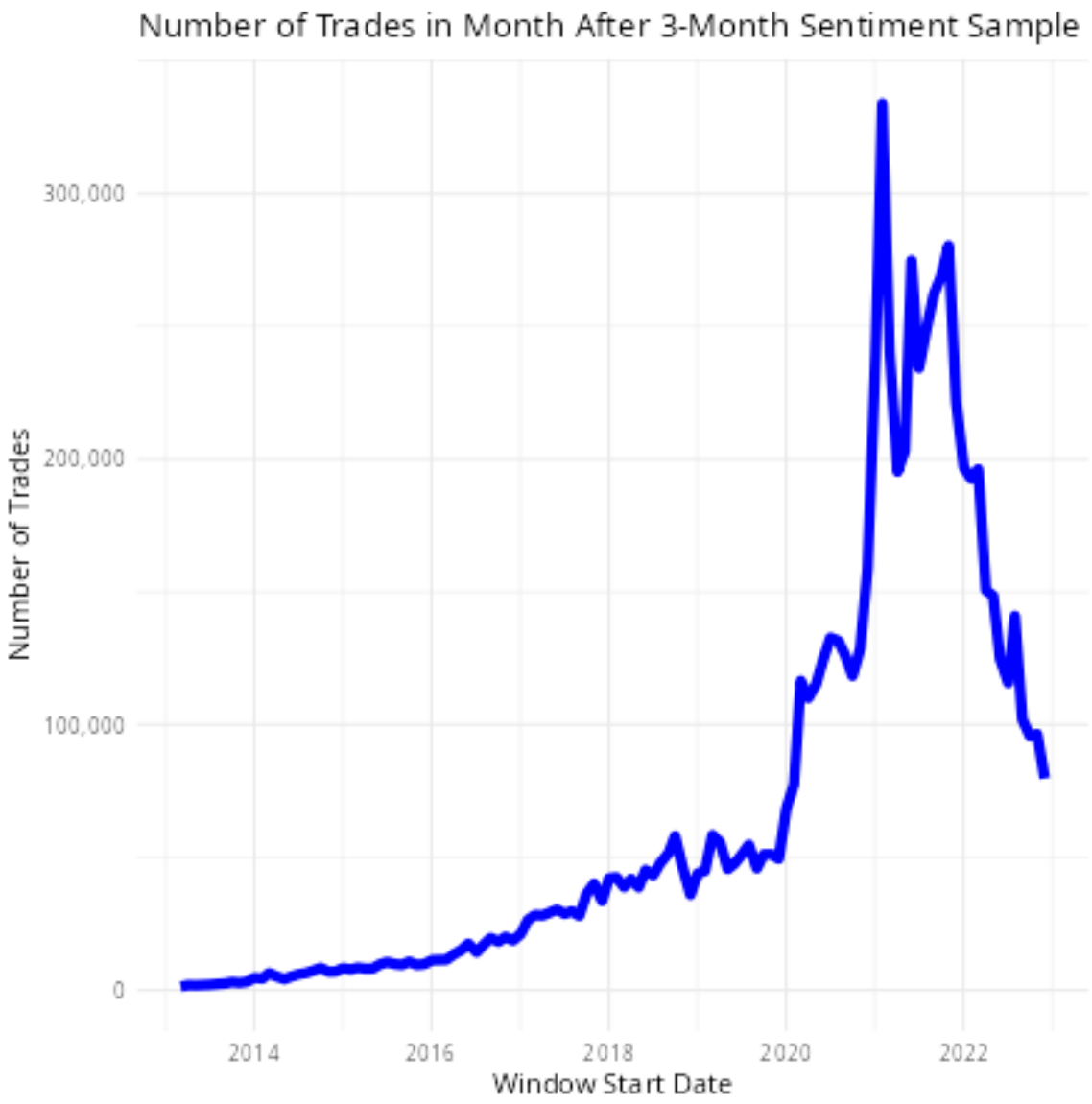


Develop Trading Strategy

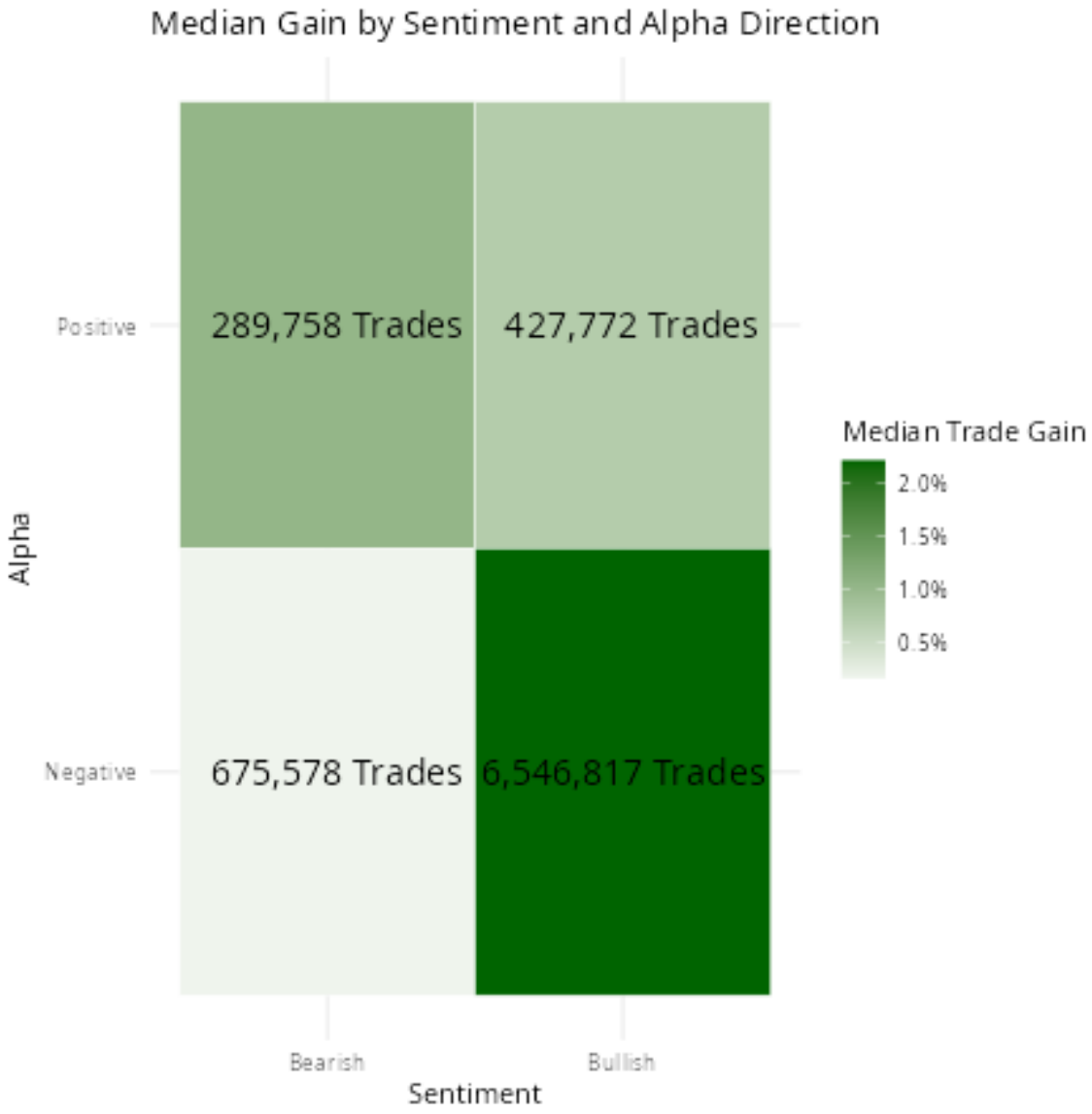
- Divide posts into 3 month samples Test for significant posters in the window.
- During the 1 month following out of sample, follow sentiment from positive alpha poster, and do the opposite from a negative alpha poster. Reverse the trade 7 days later.
- Deploy uninvested cash in SPY ETF.

This Generates a LOT of trades

Feasible for a quant shop, not us.



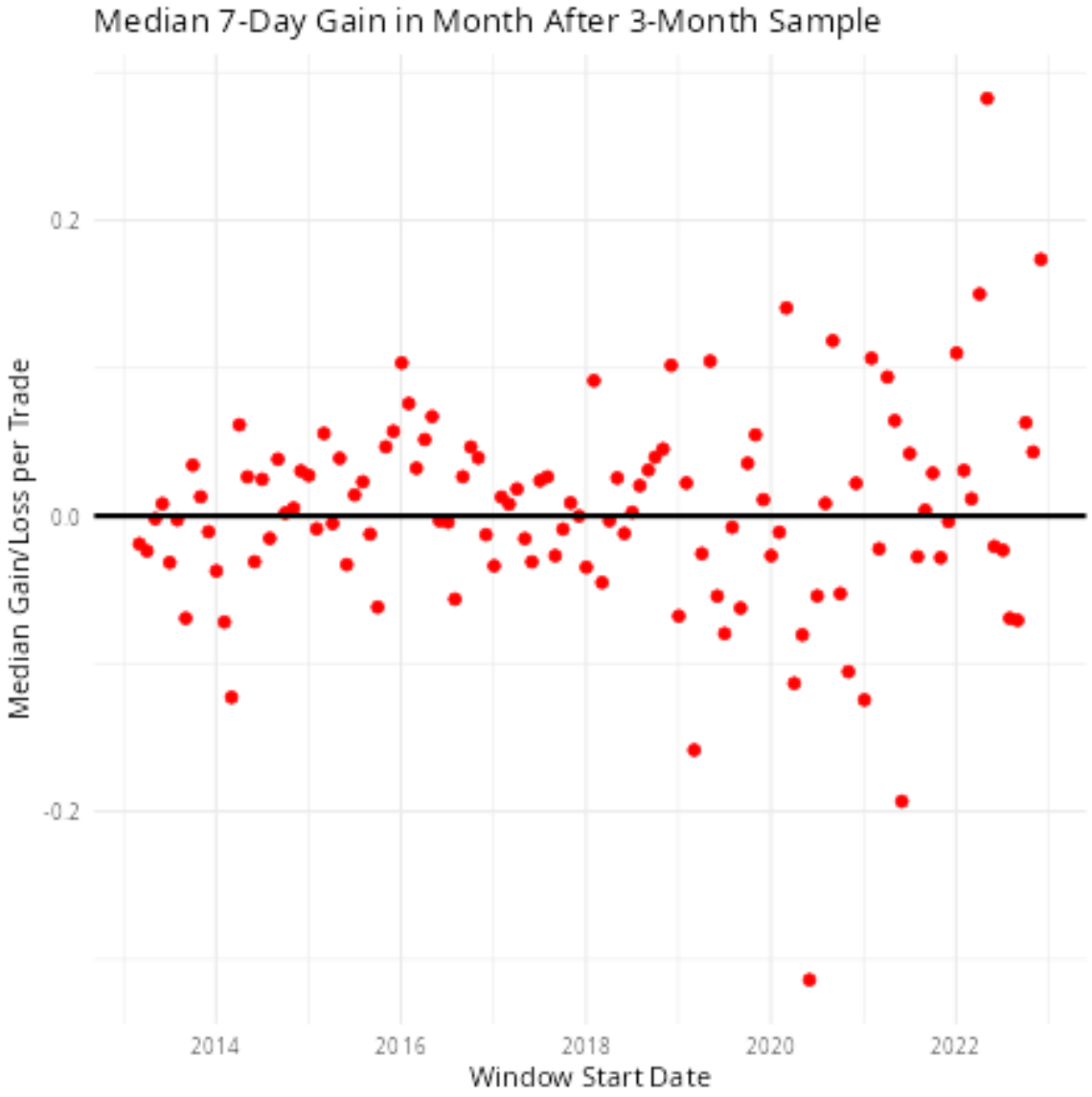
Too Bad. The Gains Look Impressive



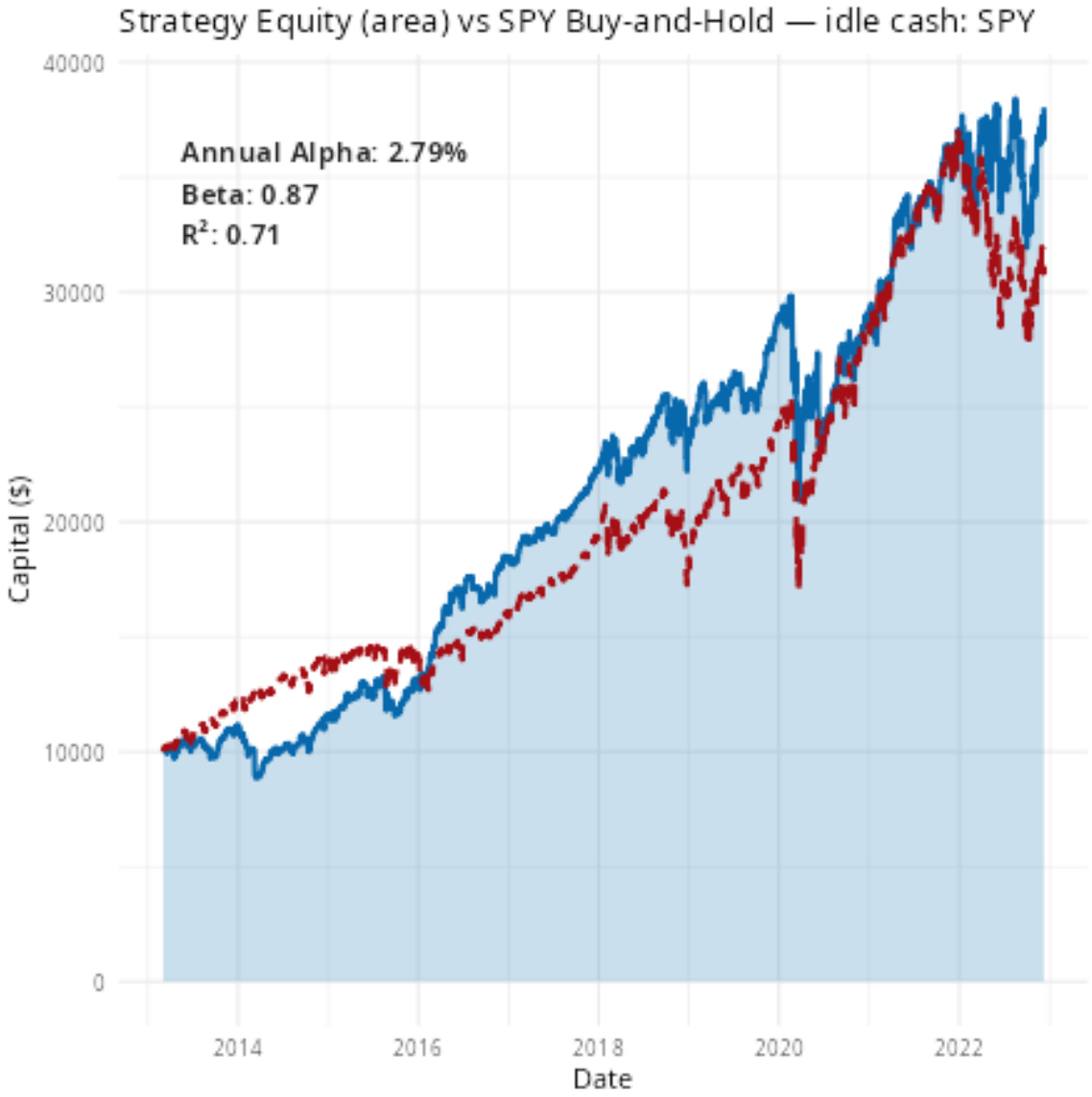
Develop Realistic Strategy for the Small Investor

- Limit trade volume to first 100 signals per month.
- Equal weight each trade.
- Long only. Note that includes buys where negative alpha posters say “Sell.”

Results!


Median 7-Day Gain in Month After 3-Month Sample
Median Gain/Loss per Trade
0.2
0.0
-0.2
2014
2016
2018
2020
2022
Window Start Date

Should I Quit My Day Job?



Caveats

- Transaction costs ignored.
- Execution Slippage ignored.
- Yahoo Finance data quality issues.
- We use a 7-day window but who knows what the poster's horizon is?
- I used AI "vibe coding" to build the trade simulator. It might be crap.

Thank You!

Bibliography

[1] X. Li, N. Al Ansari, and A. Kaufman, "StockTwits: Comprehensive records of a financial social media platform from 2008 to 2022," *Journal of Quantitative Description: Digital Media*, vol. 5, 2025, doi: 10.51685/jqd.2025.020.